

Feature inheritance rules in MedMLM – tests summary:

mlm1.use	testing basic parsing rules of mlm with empty mediators
mlm2.use	testing basic parsing rules of mlm with empty clabjects
mlm3.use	testing basic parsing rules of attribute removing and renaming
mlm3b.use	testing basic parsing rules of role removing and renaming
mlm4.use	testing basic parsing rules of assoclink

Feature testing:

1. Feature test: **Test feature navigation + end type!** **Below: no tests of type difference!**
2. Focused testing: check directly **get-attributes**, **get-navigable-ends**.
3. Integration-like testing: test feature inheritance in the context of **constraints**.

1. Inheritance of attributes in Clabjects:

Class C instance-of class D. Notation: $C : D$. attribute **a** of class C with type T: $C.a : T$.

- 1) **Default inheritance:** $D.a : T$; C has no attribute **a**.

Clabject default inheritance	test file
--	---------------------------

- 2) **Duplicated inheritance:** $C.a : T$; $D.a : T'$:

a) if $T \neq T'$, duplication error! error is thrown.

use file	fail file
----------	-----------

b) If $T == T'$, duplication error! error is thrown. **TODO:** $C.a : T$. no duplication.

use file	fail file
----------	-----------

mlm6a.use negative	when defining a clabject the child class cannot contain duplicate attributes
--	--

- 3) Inheritance with attribute removal.

there is an attribute conflict which is solved using attribute removal.

Removal in the mediator removes attribute conflict.	mlm6c.use
--	---------------------------

- 4) **Inheritance with attribute renaming:** attribute **a** of class **D** gets renamed to **a'**
 Notation: **D.a -> D.a'**
 a) no duplication.

there is an attribute conflict which is solved using attribute renaming.

Renaming in the mediator removes attribute conflict.	mlm6b.use positive
---	---------------------------------------

- b) renamed attribute name conflicts: **D.a -> D.a'**, **C.a'**, duplication error! error is thrown.

Renaming in the mediator removes attribute conflict.	mlm6e.use negative
---	---------------------------------------

- c) **D.a1**, **D.a2**, **D.a1->D.a2**, duplication error! error is thrown.

attribute duplication. Same type.	mlm6d.use negative
-----------------------------------	---------------------------------------

- d) **D.a1->a'**, **D.a2->a'**, duplication error! error is thrown.

duplicate renaming Belongs to renaming!	mlm6f.use negative
--	---------------------------------------

file: mlm3.use testCompile_mlm3_Specification	testing that the inherited attributes match the rules defined for attribute removing/renaming
file: mlm7b.use testCompile_mlm7b_Specification	testing the attribute inheritance of a 3-level MLM, when removing and renaming attributes.

2. Inheritance of roles in Clabjects:

Class C instance-of class D. Notation: **C : D**. role **r** of class H with target class L of association **assoc**: **H.r : L**.

- 1) **Assoclink inheritance overrides** role inheritance: **D.r : E** of association **assoc1**. **C** belongs to an assoclink **assoc2** of **assoc1**: **assoc2:assoc1**. **assoc2** overrides **D.r** inheritance, with or without renaming, removal, conflicts! Like “redefines”.
 a) **assoc2**: **C.r : F**, and **r → r'**: then **C.r' : F**. **r'** replaces **r**. If **r=r'**, **C.r : F**.

Assoclink_inheritance_overrides	test file
---	---------------------------

- 2) **Default inheritance:** $D.r:E$ of association **assoc1**. **C** does not have role **r**.

Default_inheritance	test file
-------------------------------------	---------------------------

- 3) **Duplicated role inheritance:** $D.r:E$ with association **assoc1**:

- a) If **C.r exists**: belongs to **in-level** association which is **not an assoclink**:
duplication conflict! error is thrown.

Duplicated_role_inheritance	test file
---	---------------------------

- b) If **C.r: F**: belongs to **inter-association**:

- i) If $E \neq F$, conflict! error is thrown.
ii) If $E = F$, conflict! error is thrown. **TODO: C.r:E.**

Duplicated_role_inheritance	test file
---	---------------------------

- c) If **C.r** belongs to **assoclink assoc**: $C.r:F \neq$.

Duplicated_role_inheritance	test file
---	---------------------------

- d) If $C.r:F$: belongs to an assoclink **assoc2** of **assoc1**: **assoc2:assoc1**:
assoc2 overrides the inherited role (like redefines!) $\rightarrow C.r:F$

Duplicated_role_inheritance	test file
---	---------------------------

- e) If $C.r:F$: belongs to an assoclink **assoc2** of **assoc3**: **assoc2:assoc3**,
assoc3 is a different association of **D**: conflict! error is thrown!

Duplicated_role_inheritance	test file
---	---------------------------

- 4) **Role inheritance with removal:** D.r :E with association **assoc1**:
- a) Removal in the mediator removes conflicts with in-level or inter-level or assoclinks of different associations!

Role_removing_inheritance	test file
---	---------------------------

- b) Removal in the mediator **does not** remove conflicts with assoclinks of same association!

- 5) **Assoclink rules:** end classes must be clabjects of end classes of upper associations



*****UNTIL HERE*****



- 6) Inheritance with role removal:
- a) If the role **r** is removed/renamed in C and C still has the association **r**, then there is no conflict.

mlm24.use negative	when removing a role in a clabject, the roles shouldn't be accessible at the current level.
mlm25.use negative	when removing a role in a clabject, the roles shouldn't be accessible at lower levels

- 7) Inheritance without role removal or renaming.

mlm2.use	testing that the inherited roles match the rules defined for role inheriting
--------------------------	--

3. Assoclinks:

mlm5a.use negative	when defining an assoclink, both ends of the related association need to have a defined clobject.
mlm5b.use negative	when defining an assoclink, the associations must exist in the MLM

4. Test design:

- 1) Complicating the model with many levels and combining previous cases.
- 2) Testing in the context of inter-constraints: either acceptance or rejection of the file containing the inter-constraint that refers to inheritance.

mlm8.use	when defining a clobject, all the attributes and roles are inherited and are accessible through inter-constraints
mlm9.use negative	when defining a clobject, all the roles are inherited but are not accessible through local constraints
mlm10.use negative	when defining a clobject, all the attributes are inherited but are not accessible through local constraints